UNDERWATER ACOUSTIC TARGET RECOGNITION WITH REAL-TIME SIGNAL VISUALIZATION

PROJECT REPORT

Submitted by

S. DAVID VINAYAK

ID No : S210118

in partial fulfillment of the requirements
for the award of the Degree of

Bachelor of Technology

In

Computer Science and Engineering

RAJIV GANDHI UNIVERSITY OF KNOWLEDGE TECHNOLOGIES

Srikakulam



NSTL-DRDO

Under the Guidance of

Mr. V. Selvaganesan

Scientist – 'G'

DECLARATION

I, **S. DAVID VINAYAK**, hereby declare that the project report entitled "**Underwater Acoustic Target Recognition with Real-Time Signal Visualization**" submitted in partial fulfilment of the requirements for the award of the degree of Bachelor of Technology in Computer Science and Engineering is a bona fide work done by me under the supervision of Mr. V. Selvaganesan, Scientist – 'G', Naval Science and Technological Laboratory (NSTL), DRDO, Visakhapatnam. This submission represents my ideas in my own words and where ideas or words of others have been included, I have adequately and accurately cited and referenced the original sources. I also declare that I have adhered to the ethics of academic honesty and integrity and have not misrepresented or fabricated any data, idea, fact, or source in my submission. I understand that any violation of the above will be a cause for disciplinary action by the institute and the University and can also evoke penal action from the sources which have thus not been properly cited or from whom proper permission has not been obtained. This report has not previously formed the basis for the award of any degree, diploma, or similar title of any other University.

Place: Visakhapatnam **S. DAVID VINAYAK**

Date: 30-10-2026 S210118

ACKNOWLEDGEMENT

I am pleased to present my project report, titled "Attention-Based Underwater Acoustic Target Recognition with Real-Time Signal Visualization," which reflects my dedicated efforts in developing a real-time acoustic signal visualization application and an attention-based deep learning classifier for underwater vessel recognition.

First and foremost, I extend my sincere gratitude to my esteemed project guide, Mr. V. Selvaganesan, Scientist – 'G', for providing invaluable mentorship, constant encouragement, and critical technical feedback throughout the design and development of the system. His expert guidance and insightful suggestions played a pivotal role in the successful completion of this project.

I would also like to express my profound gratitude to the Director, Naval Science and Technological Laboratory (NSTL), DRDO, as well as the Head of Human Resource Development (HRD), NSTL, for providing me with the opportunity to undertake this project work. Their constant support and encouragement gave me valuable exposure to research and development in the field of defence technology, hands-on experience with signal processing and deep learning, and an enriching professional research environment.

I am also thankful to Mr. Dhiren kumar sethi, Scientist - 'F' for his support and guidance during the course of this work.

Finally, I am deeply grateful to my family, friends, and peers for their continuous support, helpful suggestions, and steady encouragement throughout the development and debugging phases of this endeavour.

S. DAVID VINAYAK

S210118

B.Tech, Computer Science and Engineering

Abstract

Underwater acoustic signals are widely used for sonar, maritime surveillance, and marine monitoring, and making use of them requires two capabilities: observing signals as they arrive, and automatically recognising what produced them. This report presents an end-to-end underwater acoustic analysis system that provides both, developed in four steps.

The first step is a real-time signal visualisation application built in Python with PyQt5, PyQtGraph, NumPy, and Pandas. It reads prerecorded hydrophone data in small blocks to simulate live acquisition, and a rolling buffer keeps memory use constant. The application displays the time-domain waveform, a moving-average filtered signal, the FFT spectrum, and running signal statistics (mean, RMS, and peak amplitude) with no perceptible delay.

The remaining steps address Underwater Acoustic Target Recognition (UATR) on the DeepShip dataset, classifying recordings as cargo, passenger, tanker, or tug. Recordings are cut into 3-second overlapping clips and split into training, validation, and test sets by recording rather than by clip, so that near-identical clips from one recording never leak across sets. Three complementary features (MFCC, GFCC, and log-Mel spectrogram) are extracted from each clip, normalised, and fused as a three-channel input to a compact convolutional neural network. Efficient Channel Attention (ECA), which adds only 14 parameters, lets the network emphasise the most useful channels. Training uses class-weighted loss, SpecAugment data augmentation, and early stopping on validation macro-F1.

An ablation study compares a single-feature baseline, feature fusion, and fusion with attention. Test accuracy improves from 57.9% to 70.6%, macro-F1 from 0.5624 to 0.7528, and cargo recall from 0.8% to 49%, while ECA cuts cargo-to-tug errors from 72 clips to 3. Confusion between cargo and tanker remains the main limitation. The results are based on a partial release of DeepShip and a single training run. Validation on ShipsEar, transformer-based models, transfer learning, and integrating the classifier into the live visualisation application are left as future work.

Keywords: Underwater acoustics, target recognition, DeepShip, MFCC, GFCC, log-Mel spectrogram, Efficient Channel Attention, real-time visualisation.

Chapter 1

Introduction

1.1 Background

The underwater environment presents unique challenges for communication and signal analysis. Unlike terrestrial environments where radio waves are widely used, underwater systems rely primarily on acoustic waves because they can travel much longer distances with relatively lower attenuation.

Acoustic sensing is employed in numerous applications including sonar, marine biology, underwater robotics, oil exploration, and environmental monitoring. The received acoustic signals often contain valuable information about underwater objects, marine animals, ships, submarines, and environmental conditions.

Two distinct but complementary capabilities are required to make practical use of underwater acoustic data. The first is the ability to *acquire, visualize, and summarize* acoustic signals as they arrive, so that an operator or researcher can observe signal behaviour and data quality in real time. The second is the ability to *automatically interpret* what those signals represent — for instance, identifying the type of vessel producing a given radiated-noise signature, without requiring a human listener to classify every recording manually.

This project addresses both capabilities as one end-to-end system, built over a six-month internship at DRDO-NSTL and executed as a sequence of steps. It first builds the real-time acquisition-and-visualization module (Step 1, Chapter 5), then prepares the data (Step 2, Chapter 6), designs and trains the classification model (Step 3, Chapter 7), and finally evaluates it through an ablation study (Step 4, Chapter 9).

1.2 Project Execution Overview

The project was carried out as the four steps below. Each step produced something the next step depended on, and each is reported in its own chapter.

Table 1.1: The four execution steps of the project.

Step	Title	Outcome	Chapter
1	Signal acquisition and visualisation	Real-time display of waveform, FFT, and statistics	5
2	Dataset preparation and feature extraction	Leak-safe splits and fused MFCC/GFCC/log-Mel features	6
3	Model design and training	Baseline, fusion, and ECA fusion CNNs trained	7
4	Evaluation and ablation	Accuracy, macro-F1, recall, and confusion analysis	9

1.3 Problem Statement

Large underwater acoustic datasets are difficult to analyze manually. Traditional offline processing methods do not provide immediate feedback about signal quality, environmental changes, or — critically — what is actually present in the data. This project therefore addresses two linked problems, solved one after the other:

1. **Observation:** How can underwater acoustic data be acquired, buffered, processed, and visualized continuously and efficiently, simulating a real-time monitoring system, using standard Python tooling and prerecorded datasets?

2. **Interpretation:** Given such acoustic recordings, how can a vessel's class be automatically and reliably predicted from its radiated acoustic signature, using openly available training data and features/architectures suited to limited compute and limited training data?

Consequently, there is a need for a software platform capable of:

- Simulating real-time acquisition and displaying continuous waveforms
- Computing signal statistics and performing spectral analysis
- Automatically classifying the source of an acoustic recording

- Providing a user-friendly graphical interface and a reproducible, documented classification pipeline

1.4 Objectives

The primary objectives of this project, in the order they were pursued, are:

- Develop a real-time acoustic signal visualization system (Step 1).
- Read stored underwater acoustic data incrementally and simulate live data acquisition (Step 1).
- Compute statistical parameters including Mean, Root Mean Square (RMS), and Peak Amplitude (Step 1).
- Apply moving average filtering and Fast Fourier Transform analysis (Step 1).
- Design an intuitive graphical user interface using PyQt5 with minimal processing delay (Step 1).
- Build a leak-safe, reproducible data pipeline for the DeepShip UATR dataset (Step 2).
- Design and train a multi-feature fusion CNN (MFCC + GFCC + log-Mel spectrogram) for 4-class vessel classification (Step 3).
- Integrate Efficient Channel Attention (ECA) into the fusion architecture and empirically isolate its contribution via a controlled ablation study (Steps 3 and 4).
- Document an honest, reproducible account of model performance, including failure modes and limitations, suitable for both an internship deliverable and an academic capstone project (Step 4).

1.5 Scope of the Project

Step 1 covers acquisition simulation, signal visualization, basic preprocessing, FFT computation, statistical analysis, and graphical display. It does not include underwater com-

munication hardware, real-time sensor interfacing, or machine learning-based classification.

Steps 2 to 4 cover dataset preparation, feature engineering, model architecture design, training, and evaluation for 4-class vessel classification on DeepShip, with ShipsEar-based generalization testing as a planned/partially-completed extension. It does not cover deployment, real-time inference integration with the Step 1 GUI, or transformer-based architectures, though these are discussed as future work.

1.6 Data Used

The visualisation step (Step 1) used prerecorded acoustic data read in fixed-size blocks. The classification steps (Steps 2 to 4) use the DeepShip dataset for training, validation, and testing, with the ShipsEar dataset intended for cross-dataset generalization validation.

Chapter 2

Literature Survey

2.1 Introduction

Underwater acoustic signal processing has been an active area of research for several decades due to its importance in ocean exploration, marine biology, underwater communication, naval surveillance, and environmental monitoring. Acoustic waves are preferred over electromagnetic waves for underwater communication because they experience significantly lower attenuation in water and can travel over long distances. This chapter surveys both the real-time signal processing techniques relevant to Step 1 and the UATR classification literature relevant to Steps 2 to 4.

2.2 Classical Underwater Signal Processing

Traditional underwater acoustic monitoring systems mainly consisted of specialized hardware for data acquisition followed by offline signal processing using desktop computers [1]. Although these systems provided accurate analysis, they lacked real-time visualization and interactive monitoring capabilities. Time-domain statistics (mean, RMS, peak amplitude) and FFT-based frequency analysis remain foundational for characterizing acoustic signals and identifying tonal components associated with machinery noise and propeller cavitation [2].

Several commercial software packages are available for underwater signal analysis; however, many require expensive licenses and specialized hardware, making them less suitable for academic research and educational purposes [3].

2.3 Scientific Computing for Acoustic Analysis

Recent developments in scientific computing have enabled researchers to develop powerful signal processing tools using programming languages such as Python. Libraries including NumPy [4], SciPy [5], and Pandas [6] provide efficient implementations of numerical algorithms and large-dataset handling, while PyQt5 and PyQtGraph [7, 8] provide efficient, high-speed graphical visualization suitable for real-time applications. This approach substantially lowers the barrier to developing capable monitoring tools without reliance on expensive licensed software.

Real-time visualization techniques allow operators to continuously monitor incoming acoustic signals, detect anomalies, and perform immediate analysis. These techniques are particularly useful in underwater robotics, autonomous underwater vehicles (AUVs), and sonar systems.

2.4 Underwater Acoustic Target Recognition (UATR)

Classical UATR approaches relied on hand-crafted spectral or cepstral features (e.g., MFCC) paired with classical classifiers such as Gaussian Mixture Models or Support Vector Machines. More recent work has moved toward convolutional neural networks (CNNs) operating directly on time-frequency representations (spectrograms, MFCC, GFCC), which can learn discriminative patterns without manual feature engineering at the classifier stage, though feature *extraction* (which representation to feed the network) remains an active design choice.

2.4.1 Gammatone-Based Features

Gammatone Frequency Cepstral Coefficients (GFCC), derived from a gammatone auditory filterbank rather than the triangular mel filterbank used for MFCC, have been reported in prior audio classification literature to offer complementary noise robustness, motivating their inclusion alongside MFCC in multi-feature fusion schemes such as the one developed in this project.

2.4.2 Attention Mechanisms in CNNs

Channel attention mechanisms, which adaptively reweight the importance of different feature-map channels, have improved CNN performance across vision and audio tasks. Squeeze-and-Excitation (SE) networks popularized this idea using a bottleneck of two fully-connected layers; Efficient Channel Attention (ECA-Net) [9] subsequently argued that this dimensionality-reduction bottleneck is counter-productive for channel attention specifically, and proposed a lightweight 1D convolution across channels instead, achieving comparable or better performance with substantially fewer additional parameters — a property directly relevant to this project's resource-constrained (6GB GPU) training setting.

2.4.3 Multi-Feature Fusion

Combining multiple acoustic feature representations (e.g., stacking MFCC, GFCC, and spectrogram features as channels, analogous to RGB channels in natural images) has been explored as a means of providing the network with complementary information that no single feature captures alone. This project directly tests this hypothesis via ablation (Step 4).

2.4.4 Data Augmentation for Limited Audio Data

SpecAugment [10] introduced time and frequency masking directly on spectrogram-like features as a simple, effective regularization technique for speech/audio models trained on limited data, without requiring any change to the underlying audio. This technique was adopted in Step 3 to address overfitting observed when training on DeepShip's partial, recording-scarce release.

2.4.5 Transformer-Based Audio Models

Audio Spectrogram Transformer (AST)-style architectures, which apply the transformer attention mechanism directly to spectrogram patches, represent a more recent direction in audio classification, though they typically require larger training datasets and greater compute than CNN-based approaches to realize their full benefit. This project treats

AST-based experimentation as future work, given the compute and data constraints encountered during this project.

2.5 Research Developments Summary

Table 2.1: Summary of techniques surveyed and their role in this project.

Technique	**Origin / Purpose**	**Role in This Project**
Rolling buffer	Constant-memory streaming display	Step 1: real-time visualization
FFT	Frequency-domain analysis	Step 1: spectral display
MFCC	Mel-scale cepstral features	Step 2: primary feature
GFCC	Gammatone-scale cepstral features	Step 2: complementary feature
Log-Mel spectrogram	High-resolution time-frequency representation	Step 2: third fused feature
ECA Attention	Lightweight channel reweighting	Step 3: architecture component
SpecAugment	Time/frequency masking augmentation	Step 3: overfitting mitigation

2.6 Motivation and Research Gap

While both real-time acoustic visualization tools and deep-learning-based UATR classifiers exist as separate bodies of work, there is limited publicly documented work connecting the two into a single, coherent pipeline — particularly one developed under realistic student-project constraints (limited GPU, partial/evolving datasets, internship timelines). This project's contribution is therefore not a single novel algorithm, but a complete, honestly-documented, reproducible pipeline spanning both halves of the problem, with every design decision empirically justified rather than assumed.

Chapter 3

System Requirements

3.1 Hardware Requirements

Step 1's requirements reflect a signal-visualization workload (no GPU needed); Steps 2 to 4 reflect a deep learning training workload, which benefits substantially from GPU acceleration.

Table 3.1: Hardware requirements by project stage.

Component	**Step 1 (Visualization)**	**Steps 2–4 (Classification)**
Processor	Intel Core i5 or equivalent	Intel Core i5-12500H
RAM	8 GB or higher	8 GB or higher (16 GB recommended)
GPU	Not required	NVIDIA RTX 3050 Laptop GPU (6 GB VRAM)
Storage	10 GB free disk space	∼2–5 GB (datasets + checkpoints)
Operating System	Windows / Linux / macOS	Windows 11
Display	1366 × 768 or higher	1366 × 768 or higher

3.1.1 GPU Constraint and Its Influence on Design

The availability of only 6 GB of GPU VRAM during model .333 directly influenced several design decisions documented later in this report: the choice of a compact convolutional architecture (Chapter 7) over a larger or transformer-based model, the use of mixed-precision training, a conservative batch size of 16, and the preference for Efficient Channel Attention (which adds negligible parameters) over heavier attention mechanisms. This is noted here explicitly because hardware constraints are not merely an implementation detail — they shaped which architectures were feasible to evaluate at all within the project timeline.

3.2 Software Requirements

Table 3.2: Software stack by project stage.

Component	**Step 1**	**Steps 2–4**
Language	Python 3.x	Python 3.10
Environment Manager	–	Anaconda / conda
Deep Learning Framework	–	PyTorch 2.5.1 (CUDA 12.1 build)
GUI Framework	PyQt5	–
Plotting	PyQtGraph	Matplotlib
Numerical	NumPy	NumPy, SciPy
Data Handling	Pandas	Pandas
Audio Processing	–	librosa, spafe (GFCC extraction)
ML Utilities	–	scikit-learn (evaluation metrics)
IDE	VS Code / PyCharm	Anaconda Prompt / VS Code

3.3 Python Libraries Used

3.3.1 Step 1: Visualisation

- NumPy — numerical array operations
- Pandas — CSV dataset reading
- PyQt5 — graphical user interface framework
- PyQtGraph — high-speed real-time plotting
- `collections.deque` — rolling buffer implementation
- SciPy — optional alternative FFT implementation

3.3.2 Steps 2–4: Classification

- PyTorch, torchaudio — model definition, training loop, GPU acceleration
- librosa — audio loading, resampling, MFCC and log-Mel extraction
- spafe — GFCC (Gammatone Frequency Cepstral Coefficient) extraction
- NumPy, SciPy — numerical operations and signal processing primitives

- Pandas — manifest/metadata management, CSV logging
- scikit-learn — macro-F1, recall, confusion matrix computation
- Matplotlib — figure generation for this report
- PyYAML — configuration file management

Chapter 4

Theoretical Background

4.1 Introduction

This chapter establishes the theoretical foundations underlying this project. Section 4.2 covers digital signal processing concepts central to Step 1 (and to the feature extraction of Step 2). Section 4.3 covers machine learning and deep learning concepts central to the classification model of Steps 3 and 4. Readers already familiar with these fundamentals may proceed directly to Chapter 5.

4.2 Digital Signal Processing Fundamentals

4.2.1 Sampling and the Nyquist Theorem

A continuous-time acoustic signal $x(t)$ must be converted to a discrete-time sequence $x[n]$ for digital processing. This is done by sampling at a fixed rate F_s (samples per second), giving

$$x[n] = x(nT_s), \qquad T_s = \frac{1}{F_s}.$$

The Nyquist–Shannon sampling theorem states that a signal can be perfectly reconstructed from its samples only if $F_s \geq 2f_{\max}$, where $f_{\max}$ is the highest frequency component present in the signal. Sampling below this rate causes *aliasing*, where high-frequency content is misrepresented as lower-frequency artifacts. This theorem underlies the choice of sampling rate at both stages: the Step 1 prerecorded datasets were sampled at $F_s = 50{,}000$ Hz, while Step 2 resamples all audio to $F_s = 16{,}000$ Hz — a rate standard

in audio classification literature and sufficient to capture the frequency range relevant to vessel radiated-noise signatures, which are dominated by low-to-mid frequency machinery and propeller harmonics.

4.2.2 Time-Domain versus Frequency-Domain Representation

A signal may be represented either as amplitude over time (time domain) or as the distribution of energy across frequency components (frequency domain). The two are related by the Fourier Transform. Time-domain representation is intuitive for observing transient events and overall signal envelope (used directly in the Step 1 waveform display), while frequency-domain representation reveals periodic or tonal structure — such as a vessel's propeller shaft rate or engine firing frequency — that may not be visually obvious in the time domain.

4.2.3 Discrete Fourier Transform and the FFT

The Discrete Fourier Transform (DFT) converts a finite sequence of N time-domain samples into N frequency-domain coefficients:

$$X(k) = \sum_{n=0}^{N-1} x(n)\, e^{-j2\pi kn/N}, \qquad k = 0, 1, \ldots, N-1.$$

Direct computation of the DFT requires $O(N^2)$ operations. The Fast Fourier Transform (FFT) algorithm computes the same result in $O(N \log N)$ operations by recursively decomposing the DFT, making real-time frequency analysis computationally feasible — this is the algorithm underlying both the live spectrum display of Step 1 (via `numpy.fft.rfft`) and the spectral feature extraction used in Step 2.

4.2.4 Windowing

When the FFT is applied to a finite-length segment of a longer signal, implicitly treating that segment as periodic, discontinuities at the segment boundaries introduce spectral leakage — spurious energy spread across frequencies not actually present in the signal. A window function $w(n)$ (e.g., Hamming, Hanning) is applied to taper the segment's edges

toward zero before the FFT, reducing this leakage:

$$x_w(n) = x(n) \cdot w(n).$$

Step 2's feature extraction uses a Hamming window (via `spafe`'s `SlidingWindow`) for GFCC computation and librosa's default windowing for MFCC and log-Mel extraction.

4.2.5 Short-Time Fourier Transform and Spectrograms

A single FFT describes the frequency content of an entire signal segment, losing information about *when* each frequency component occurs. The Short-Time Fourier Transform (STFT) addresses this by computing the FFT over a sliding window across the signal:

$$X(m,k) = \sum_n x(n)\, w(n - mH)\, e^{-j2\pi kn/N},$$

where H is the hop length (step size between windows) and m indexes the time frame. The magnitude $|X(m,k)|^2$ forms a *spectrogram* — a 2D time-frequency representation. This project's The Step 2 feature extraction uses $N = 1024$ (FFT size), $H = 256$ (hop length), yielding 188 time frames per 3-second clip at 16 kHz.

4.2.6 Mel Scale and MFCC

Human auditory perception of pitch is approximately logarithmic rather than linear; the mel scale models this relationship:

$$\text{mel}(f) = 2595 \log_{10}\left(1 + \frac{f}{700}\right).$$

A mel-spectrogram is computed by passing the STFT's power spectrum through a bank of triangular filters spaced uniformly on the mel scale, then taking the logarithm (producing the log-Mel spectrogram used directly as one of the three fused features of Step 2). Mel-Frequency Cepstral Coefficients (MFCC) are obtained by further applying a Discrete Cosine Transform (DCT) to the log-Mel filterbank energies, which decorrelates the filterbank outputs and compresses the representation into a small number of coefficients (40, in this project) capturing the overall spectral envelope shape.

4.2.7 Gammatone Filterbank and GFCC

An alternative to the mel filterbank is the gammatone filterbank, whose impulse response,

$$g(t) = t^{n-1} e^{-2\pi bt} \cos(2\pi f_c t + \phi),$$

is motivated by physiological models of the human cochlea's frequency response, rather than the purely perceptual mel scale. Gammatone Frequency Cepstral Coefficients (GFCC) are computed analogously to MFCC but using this filterbank instead. Prior literature suggests GFCC can offer improved robustness in noisy conditions relative to MFCC, motivating its inclusion as a complementary feature in this project's fusion scheme (Section 6.5).

4.2.8 Statistical Signal Descriptors

In addition to time-frequency representations, simple statistical descriptors summarize a signal's overall characteristics. For a signal of N samples x_i:

$$\mu = \frac{1}{N}\sum_{i=1}^{N} x_i, \qquad \text{RMS} = \sqrt{\frac{1}{N}\sum_{i=1}^{N} x_i^2}, \qquad \text{Peak} = \max_i |x_i|.$$

These are computed continuously in Step 1 to provide a quick, interpretable summary of signal energy and amplitude, independent of any frequency-domain analysis.

4.3 Machine Learning and Deep Learning Fundamentals

4.3.1 Supervised Learning

The classification task of Steps 2 to 4 is an instance of supervised learning: given a labeled training set $\{(x_i, y_i)\}_{i=1}^{N}$, where x_i is an input feature representation and $y_i \in \{\text{cargo}, \text{passenger}, \text{tanker}, \text{tug}\}$ is the corresponding class label, the goal is to learn a function $f_\theta(x)$, parameterized by weights θ, that generalizes to unseen inputs. Generalization — performance on data not seen during training — is the central concern of the

classification methodology, motivating the leak-safe recording-level train/validation/test split described in Section 6.3.2.

4.3.2 Neural Networks and Backpropagation

A neural network computes $f_\theta(x)$ as a composition of layers, each applying a linear transformation followed by a nonlinear activation function:

$$h^{(l)} = \sigma\left(W^{(l)} h^{(l-1)} + b^{(l)}\right),$$

where $W^{(l)}, b^{(l)}$ are the weights and biases of layer l, and σ is a nonlinearity (e.g., ReLU: $\sigma(z) = \max(0, z)$). Training minimizes a loss function $\mathcal{L}(\theta)$ measuring the discrepancy between predictions and true labels, via gradient descent: weights are updated as $\theta \leftarrow \theta - \eta \nabla_\theta \mathcal{L}$, where the gradient is computed efficiently for all layers simultaneously using the backpropagation algorithm (reverse-mode automatic differentiation via the chain rule).

4.3.3 Convolutional Neural Networks

Convolutional Neural Networks (CNNs) are well suited to time-frequency representations (spectrograms, MFCC, GFCC) because they exploit local spatial structure — nearby time frames and nearby frequency bins are more likely to be related than distant ones — using far fewer parameters than a fully-connected network would require for the same input size.

Convolution

A 2D convolutional layer applies a small learnable kernel K across the input feature map:

$$(x * K)(i, j) = \sum_m \sum_n x(i + m, j + n)\, K(m, n).$$

Each kernel learns to detect a specific local pattern (e.g., a particular time-frequency texture); a layer typically contains many kernels, producing multiple output channels.

Batch Normalization

Batch normalization normalizes each layer's activations to zero mean and unit variance (per mini-batch, per channel) before applying a learned scale and shift:

$$\hat{h} = \frac{h - \mu_{\text{batch}}}{\sqrt{\sigma^2_{\text{batch}} + \epsilon}}, \qquad y = \gamma \hat{h} + \beta.$$

This stabilizes and accelerates training by reducing internal covariate shift, and is used after every convolutional layer in both the baseline and fusion models developed in Step 3.

Pooling

Max-pooling downsamples a feature map by taking the maximum value within each local window (e.g., 2×2), reducing spatial resolution and parameter count in subsequent layers while retaining the most salient activations. Both models in this project use 2×2 max-pooling after each convolutional block.

Global Average Pooling

Rather than flattening a feature map directly into a fully-connected layer (which fixes the required input spatial size), global average pooling reduces each channel to a single value by averaging across all spatial positions, making the model robust to minor variation in input size and substantially reducing the parameter count of the classifier head.

4.3.4 Loss Functions and Class Weighting

For multi-class classification, the cross-entropy loss is standard:

$$\mathcal{L} = -\sum_{c=1}^{C} y_c \log(\hat{y}_c),$$

where y_c is the one-hot true label and $\hat{y}_c$ is the predicted probability for class c (via softmax). Under class imbalance — as in this project's DeepShip data, where recording counts differ substantially across classes — the loss may be modified with per-class weights

w_c:

$$\mathcal{L}_{\text{weighted}} = -\sum_{c=1}^{C} w_c \, y_c \log(\hat{y}_c),$$

increasing the penalty for misclassifying under-represented classes. Section 7.5 details how these weights were computed in this project, and why weighting by recording count rather than clip count proved important.

4.3.5 Optimization

Gradient Descent Variants

Stochastic Gradient Descent (SGD) updates weights using gradients computed on small mini-batches rather than the full dataset, trading some gradient accuracy for substantially faster and more memory-efficient training. Adam (Adaptive Moment Estimation) extends SGD by maintaining per-parameter adaptive learning rates based on running estimates of the first and second moments of the gradient:

$$m_t = \beta_1 m_{t-1} + (1-\beta_1) g_t, \qquad v_t = \beta_2 v_{t-1} + (1-\beta_2) g_t^2,$$

$$\theta_t = \theta_{t-1} - \eta \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon}.$$

AdamW, used throughout training, modifies Adam by decoupling weight decay (L2 regularization) from the gradient-based update, which has been shown empirically to improve generalization relative to standard Adam with L2 regularization folded into the loss.

Learning Rate Scheduling

A cosine annealing schedule decays the learning rate smoothly following a cosine curve over training:

$$\eta_t = \eta_{\min} + \frac{1}{2}(\eta_{\max} - \eta_{\min}) \left(1 + \cos\left(\frac{t}{T}\pi\right)\right),$$

allowing larger updates early in training (faster initial progress) and smaller, more refined updates later (stable convergence). This schedule was used for all training runs.

Mixed-Precision Training

Mixed-precision training performs most computation in 16-bit floating point (half precision) rather than the standard 32-bit, roughly halving memory usage and often accelerating computation on compatible GPUs, while maintaining a 32-bit master copy of weights for numerically sensitive updates. This was essential to the feasibility of training given the 6 GB VRAM constraint of the available GPU.

4.3.6 Regularization and Overfitting

A model overfits when it achieves low training loss but fails to generalize to unseen data — learning to memorize training-set-specific patterns rather than the underlying class-discriminative structure. This was observed directly during baseline development (training macro-F1 approaching 0.9+ while validation macro-F1 remained low and volatile), motivating two regularization strategies:

Dropout

Dropout randomly zeroes a fraction p of a layer's activations during each training step, preventing the network from relying too heavily on any single unit and acting as an implicit ensemble of sub-networks. Used at $p = 0.3$ in this project's classifier heads.

Data Augmentation (SpecAugment)

SpecAugment [10] applies random time and frequency masking directly to spectrogram-like features, forcing the model to learn representations robust to missing information rather than memorizing exact recording-specific patterns. This was found empirically necessary in Step 3 given the small number of distinct source recordings available in the partial DeepShip release.

Early Stopping

Training is halted once validation performance fails to improve for a fixed number of epochs (patience), preventing continued training from degrading generalization even as training loss continues to fall. A patience of 10 epochs, monitored on validation macro-F1, was used throughout training.

4.3.7 Attention Mechanisms

Motivation

A standard CNN treats all channels of a feature map as equally important when passing them to subsequent layers. Channel attention mechanisms instead learn to adaptively reweight channels based on their content, allowing the network to emphasize more informative channels (e.g., a particular feature-map channel that happens to capture a discriminative frequency band) and suppress less useful ones.

Squeeze-and-Excitation (SE) Attention

SE-Net computes channel attention via: (1) global average pooling to obtain one value per channel, (2) a bottleneck of two fully-connected layers (reducing then restoring dimensionality) to model channel interdependencies, and (3) a sigmoid to produce per-channel weights in $[0, 1]$, which rescale the original feature map.

Efficient Channel Attention (ECA)

ECA-Net [9] argues that SE's dimensionality-reduction bottleneck is unnecessary and in fact harmful for channel attention, since it discards direct correspondence between a channel and its own weight computation. ECA instead applies global average pooling followed directly by a 1D convolution across the channel dimension (no dimensionality change), with kernel size k chosen adaptively based on channel count C:

$$k = \left| \frac{\log_2(C)}{\gamma} + \frac{b}{\gamma} \right|_{\text{odd}},$$

followed by a sigmoid to produce the final per-channel attention weights. This design captures local cross-channel interaction (channels close in index interact more directly) with far fewer added parameters than SE — in this project's configuration, only 14 additional parameters across the entire network (Section 7.3).

4.3.8 Evaluation Metrics

Precision, Recall, and F1-Score

For a given class c, with true positives TP_c, false positives FP_c, and false negatives FN_c:

$$\text{Precision}_c = \frac{TP_c}{TP_c + FP_c}, \qquad \text{Recall}_c = \frac{TP_c}{TP_c + FN_c}, \qquad F1_c = \frac{2 \cdot \text{Precision}_c \cdot \text{Recall}_c}{\text{Precision}_c + \text{Recall}_c}.$$

Recall measures the fraction of a class's true instances correctly identified — directly relevant when a model systematically fails on one class (as the baseline model did on Cargo), since this would be invisible in an aggregate accuracy figure if other classes are classified well.

Macro-Averaging and Why It Matters Here

Macro-F1 averages the F1-score across classes *without* weighting by class frequency:

$$\text{Macro-F1} = \frac{1}{C} \sum_{c=1}^{C} F1_c.$$

This is the primary metric used throughout training, specifically because DeepShip's class distribution is imbalanced: a model that performs well on common classes (Tanker) but poorly on rare or hard classes (Cargo, Tug) would still achieve a deceptively high overall accuracy, while macro-F1 penalizes this unevenness directly — which is exactly the failure mode observed and corrected over the course of this project's ablation study.

Confusion Matrix

A confusion matrix M, where M_{ij} counts instances of true class i predicted as class j, provides a complete, non-aggregated view of model errors, revealing *which* classes are confused with which — the primary tool used in Chapter 9 to diagnose and explain the Cargo–Tanker confusion pattern observed throughout this project.

4.4 Chapter Summary

This chapter established the DSP foundations (sampling, FFT, windowing, mel/gammatone filterbanks) underlying the real-time visualization of Step 1 and the feature extraction of

Step 2, and the deep learning foundations (CNNs, attention, optimization, regularization, evaluation metrics) underlying the classification model of Steps 3 and 4. The remaining chapters apply these concepts in the order the project was executed: Chapter 5 describes Step 1, and Chapters 6–9 describe Steps 2 to 4.

Chapter 5

Step 1: Real-Time Acoustic Signal Acquisition and Visualization

5.1 Overview

The first step of the project, carried out at the Naval Science and Technological Laboratory (NSTL), Defence Research and Development Organisation (DRDO), developed a software application simulating real-time underwater acoustic signal acquisition, visualization, and basic analysis using prerecorded datasets. Rather than interfacing with physical hydrophone hardware, the application reads stored acoustic samples in fixed-size blocks and continuously updates its graphical display, simulating live data acquisition while remaining fully reproducible on standard laptop/desktop hardware without specialized equipment.

This chapter presents the methodology, system architecture, implementation, and results of Step 1. The underlying DSP theory (FFT, windowing, statistical descriptors) was presented in Chapter 4 and is referenced rather than re-derived here.

5.2 Methodology and Workflow

The processing pipeline consists of seven sequential stages, summarized in Figure 5.1: reading the acoustic dataset, simulating real-time acquisition via timer-based block reads, updating a rolling buffer, computing signal statistics, applying moving-average filtering, performing FFT analysis, and updating the graphical display.

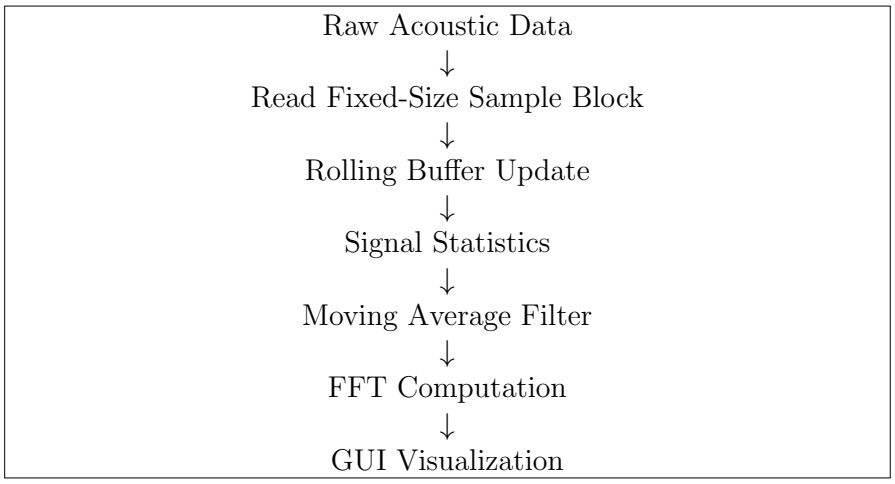


Figure 5.1: Step 1 processing workflow.

5.2.1 Reading the Dataset

The acoustic dataset is read using the Pandas library, with the first column representing time values and the second column representing signal amplitudes.

Listing 5.1: Reading the acoustic dataset.

```
df = pd.read_csv(filename, header=None)

time = df.iloc[:, 0].values
signal = df.iloc[:, 1].values
```

5.2.2 Real-Time Acquisition Simulation

Rather than displaying all samples simultaneously, the application reads a fixed number of samples during every timer event. With a block size $N = 1000$ samples and sampling frequency $F_s = 50{,}000$ Hz (Section 4.2), the update interval is

$$T = \frac{N}{F_s} = \frac{1000}{50000} = 0.02 \text{ s} = 20 \text{ ms}.$$

The GUI therefore simulates live acquisition by refreshing its display every 20 milliseconds — fast enough to appear continuous to a human observer, while remaining computationally lightweight.

5.2.3 Rolling Buffer

A rolling buffer retains only the most recent one second of acoustic data. At $F_s = 50{,}000$ Hz, one second corresponds to 50,000 samples. The buffer automatically discards the oldest samples as new samples arrive, maintaining a constant memory footprint regardless of total dataset size or how long the application has been running.

Advantages of the Rolling Buffer Design

- Constant memory utilization, independent of total recording length
- Smooth, uninterrupted visualization
- Fast updates with minimal per-frame overhead
- Efficient simulation of real-time processing
- Reduced computational complexity relative to re-processing the full dataset on every update

5.3 System Architecture

The application consists of seven functional modules operating together: a Data Reader, a Real-Time Acquisition Simulator, a Rolling Buffer, a Signal Processing module, a Statistical Analysis module, an FFT Analysis module, and the Graphical User Interface itself.

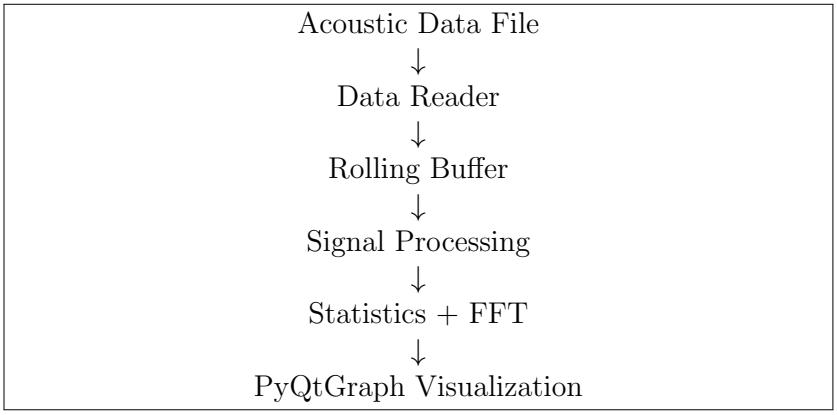


Figure 5.2: Step 1 overall system architecture.

5.3.1 Working Principle

On startup, the application loads the acoustic data file. Rather than displaying the complete dataset, the software reads a fixed number of samples during each timer interval and appends them to the rolling buffer, which retains only the latest one second of data. This buffered signal is then processed to compute statistical parameters, smoothed via moving-average filtering, and transformed to the frequency domain via FFT. The processed results are rendered continuously using PyQtGraph.

5.4 Implementation

5.4.1 Development Environment

The software was implemented in Python, chosen for its extensive scientific computing ecosystem and comparatively rapid development cycle. PyQt5 provided the graphical interface framework, and PyQtGraph provided high-speed, interactive plotting suitable for continuously updating displays.

5.4.2 Rolling Buffer Implementation

A `deque` (double-ended queue) from Python's standard library implements the rolling buffer, with a fixed `maxlen` equal to one second of data at the sampling rate.

Listing 5.2: Rolling buffer implementation.

```
from collections import deque

buffer = deque(maxlen=50000)   # one second at 50 kHz
```

During every timer event, the next chunk of samples is appended; older samples are automatically discarded once the buffer reaches its maximum capacity, with no manual bookkeeping required.

Listing 5.3: Updating the rolling buffer on each timer tick.

```
buffer.extend(chunk_signal)
```

5.4.3 Real-Time Plotting

The waveform is updated continuously using PyQtGraph's `setData` method, which efficiently redraws only what has changed rather than re-rendering the entire plot from scratch — essential for maintaining smooth visual performance at a 20 ms update interval.

Listing 5.4: Updating the live waveform display.

```
curve.setData(time_buffer, signal_buffer)
```

5.4.4 Signal Statistics, Filtering, and FFT

At each update, the application computes the signal statistics and spectral representation defined in Section 4.2: mean, RMS, and peak amplitude over the buffered signal; a moving-average-smoothed waveform via `numpy.convolve`; and an FFT-derived frequency spectrum via `numpy.fft.rfft`.

Listing 5.5: Moving average filter.

```
avg = np.convolve(signal,
                  np.ones(window) / window,
                  mode='valid')
```

Listing 5.6: FFT-based frequency-domain analysis.

```
fft_data = np.fft.rfft(signal)

frequency = np.fft.rfftfreq(len(signal), d=1/Fs)
```

5.4.5 Graphical User Interface

The GUI presents four coordinated views: the time-domain waveform, the moving-average-smoothed waveform, the FFT-derived frequency spectrum, and the live-updating signal statistics (mean, RMS, peak), alongside basic playback controls (pause/resume acquisition, reset display zoom). The interface updates automatically without perceptible delay, providing continuous visualization of the simulated incoming signal.

5.5 Results and Discussion

5.5.1 Time-Domain Visualization

The time-domain waveform updates continuously without noticeable delay. The rolling buffer ensures only the latest one second of data is displayed at any time, maintaining a constant memory footprint regardless of how long the application runs. This allows direct observation of signal variations, transient events, and amplitude changes as they occur.

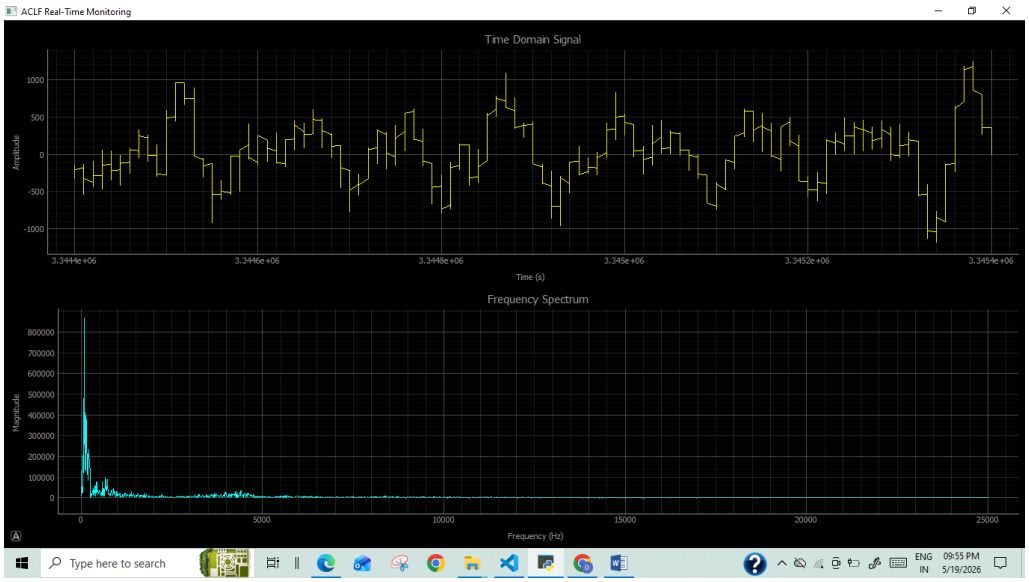


Figure 5.3: Step 1: Time-domain waveform visualization (oscilloscope-style display).

5.5.2 Frequency-Domain Analysis

The FFT module converts the buffered time-domain signal into the frequency domain, allowing dominant frequency components to be identified. This is particularly useful for detecting tonal signals, mechanical vibrations, underwater machinery noise, and biological acoustic emissions — a capability directly motivating the later use of frequency-domain features in Step 2 (MFCC, GFCC, log-Mel spectrograms) for automated classification.

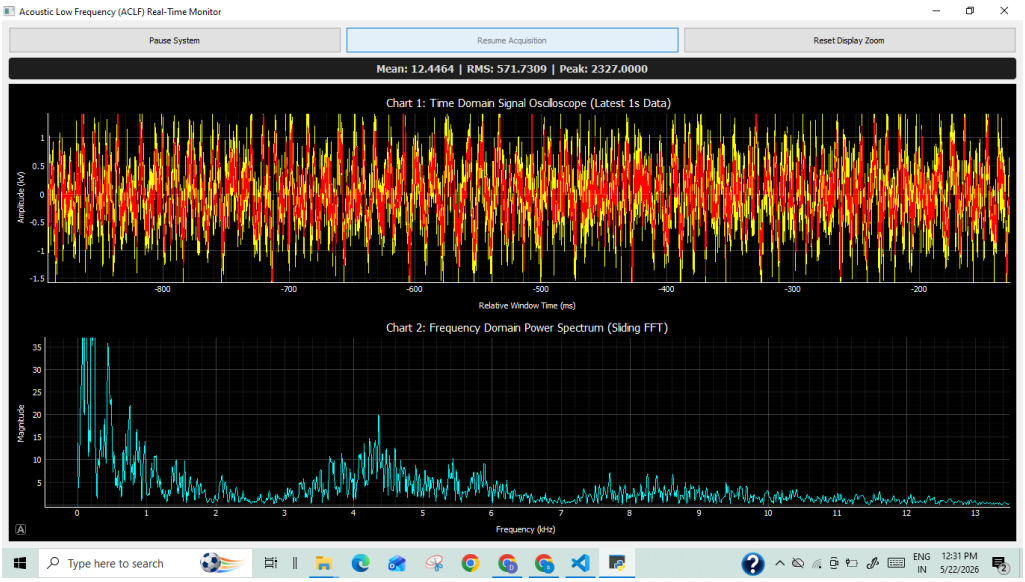


Figure 5.4: Step 1: FFT-derived frequency spectrum, computed on a sliding window of the rolling buffer.

5.5.3 Moving Average Output

The moving-average filter successfully smooths the displayed waveform, reducing short-term fluctuations while preserving the signal's overall trend. This assists in observing slow variations in the acoustic data and reduces the visual influence of high-frequency noise on the primary waveform display.

5.5.4 Signal Statistics

The application continuously computes Mean, RMS, and Peak Amplitude, providing a quick, interpretable summary of signal characteristics useful for monitoring changes in underwater acoustic conditions without requiring full spectral analysis for every observation.

5.5.5 Performance Summary

Table 5.1: Step 1 application performance characteristics.

Characteristic	**Result**
Display update interval	20 ms (50 Hz refresh)
Buffer memory footprint	Constant (1 second of data, regardless of file size)
Visualization latency	Not perceptible to the user
Computational overhead	Low (suitable for standard laptop hardware)

The developed application achieved smooth, low-latency visualization of both the time-domain waveform and FFT-derived frequency spectrum, with constant memory usage regardless of underlying dataset size — directly validating the rolling-buffer design choice made in Section 4.2.

5.6 Step 1 Summary and Link to Step 2

Step 1 established that Python tooling (PyQt5, PyQtGraph, NumPy) can deliver responsive, low-overhead real-time acoustic visualization without specialized hardware. However, the analysis in this step was fundamentally *descriptive*: it summarized and displayed signal properties (mean, RMS, dominant frequencies) but made no judgment about what the signal represented — what type of vessel, if any, produced it.

This observation directly motivated the next step, described from Chapter 6: having built the infrastructure to *observe* underwater acoustic signals in real time, the natural next step was to build the intelligence to *automatically interpret* them. The classification work of Steps 2 to 4 therefore builds directly on the signal-processing foundation laid in Step 1.

Chapter 6

Step 2: Dataset Preparation and Feature Extraction

6.1 Introduction

Having established in Step 1 that underwater acoustic signals can be acquired and visualized in real time, this step addresses the complementary problem of automatically determining *what* a given acoustic recording represents. This chapter describes the datasets used, the preprocessing pipeline applied to them, and the feature extraction and fusion scheme that forms the input to the classification model described in Chapter 7.

6.2 Datasets

6.2.1 DeepShip (Primary Dataset)

DeepShip is a publicly available dataset of underwater radiated ship-noise recordings, organized into four vessel-type classes: *cargo*, *passenger*, *tanker*, and *tug*. This project used a partial public release of DeepShip (hosted on GitHub), rather than the complete dataset, due to file-hosting size restrictions on the source repository. The recording counts used are summarized in Table 6.1.

Table 6.1: DeepShip recordings used, by class (partial public release).

Class	Total Recordings	Train	Validation	Test
Cargo	12	8	2	2
Passenger	20	14	3	3
Tanker	28	20	4	4
Tug	3	1	1	1

In total, the 63 recordings produced 1382, 710, 921, and 525 training clips, 260, 296, 334, and 125 validation clips, and 390, 111, 368, and 134 test clips for cargo, passenger, tanker, and tug respectively.

A known limitation of this partial release is the very small number of Tug recordings (3 total), which limits the statistical reliability of Tug-specific metrics; this is discussed further in Chapter 12.

6.2.2 ShipsEar (Secondary Dataset, Generalization Validation)

ShipsEar is a publicly available underwater noise database comprising 90 recordings across 11 original vessel-type categories, intended in this project's design as a secondary dataset for cross-dataset generalization validation — testing whether a model trained on DeepShip generalizes to acoustically similar but distinct recordings from an entirely different data source.

In total, the 63 recordings produced 1382, 710, 921, and 525 training clips, 260, 296, 334, and 125 validation clips, and 390, 111, 368, and 134 test clips for cargo, passenger, tanker, and tug respectively.

6.3 Preprocessing Pipeline

6.3.1 Resampling and Segmentation

All audio is resampled to 16 kHz, mono (Section 4.2 discusses the sampling-rate trade-offs involved). Recordings are then segmented into fixed-length 3-second clips with 50% overlap between consecutive clips, a common windowing choice in UATR literature that balances providing the model with sufficient temporal context per example against generating enough training examples from a limited number of source recordings.

6.3.2 Leak-Safe Train/Validation/Test Splitting

A critical methodological decision in this project is that the train/validation/test split is performed at the **recording level**, not the clip level: all clips derived from a given source recording are assigned entirely to a single split. This is essential because the 50% clip overlap means adjacent clips from the same recording are substantially similar; a naive random split by clip would allow near-duplicate segments of the same recording to appear in both the training set and the test set, artificially inflating test accuracy by letting the model partially "recognize" a recording it was effectively trained on. This is a documented pitfall in UATR literature and was deliberately avoided throughout this project.

Given that some classes have very few source recordings (notably Tug, with as few as 3 total), a single global stratified split was found to be unreliable — in early development, this approach occasionally produced a split where a class had zero representation in one partition. A manual per-class allocation procedure was used instead, guaranteeing every class receives at least one recording in each split wherever the total recording count allows.

6.3.3 Resulting Clip Distribution

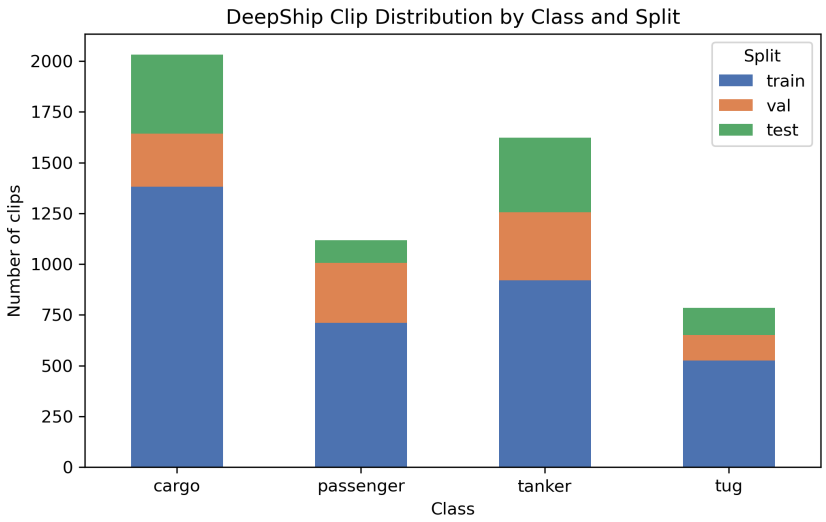


Figure 6.1: Distribution of segmented clips across the four DeepShip classes, by train/validation/test split.

As Figure 6.1 shows, class imbalance exists at both the recording level and, consequently, the clip level, though not in direct proportion — a subtlety that significantly influenced

the class-weighting strategy described in Section 7.5.

6.4 Feature Extraction

Three acoustic feature representations, each grounded in the DSP theory of Section 4.2, are computed per 3-second clip:

- **MFCC** (40 coefficients) — captures the spectral envelope on the perceptually motivated mel frequency scale.
- **GFCC** (40 coefficients) — uses a gammatone filterbank, motivated by auditory physiology, offering complementary noise robustness relative to MFCC.
- **Log-Mel spectrogram** (40 mel bins) — a higher-resolution time-frequency representation prior to cepstral compression via DCT.

All three features are computed with FFT size 1024, hop length 256, and window length 1024 samples (at 16 kHz), yielding 188 time frames per 3-second clip. The log-Mel bin count was deliberately set to 40 — rather than the more conventional 128 bins typically used for log-Mel spectrograms — specifically to match MFCC and GFCC's coefficient count, enabling the three features to be stacked as channels (Section 6.5) without additional resizing logic. This is a deliberate simplicity-over-resolution trade-off, documented here as a conscious design decision rather than an oversight.

6.4.1 Normalization

Each feature is normalized *per coefficient* (i.e., per frequency bin, independently across the time axis) to zero mean and unit variance — a form of cepstral mean-variance normalization (CMVN). This proved necessary during development: an earlier, simpler *global* normalization (computing a single mean and standard deviation across the entire feature matrix) allowed the 0th coefficient — which encodes overall signal energy and has substantially larger magnitude than the remaining coefficients — to dominate the normalization statistics. This visually and numerically suppressed the genuine discriminative variation present in the other coefficients, confirmed via a dedicated feature-sanity-check visualization performed before any model training began (Figure 6.2).

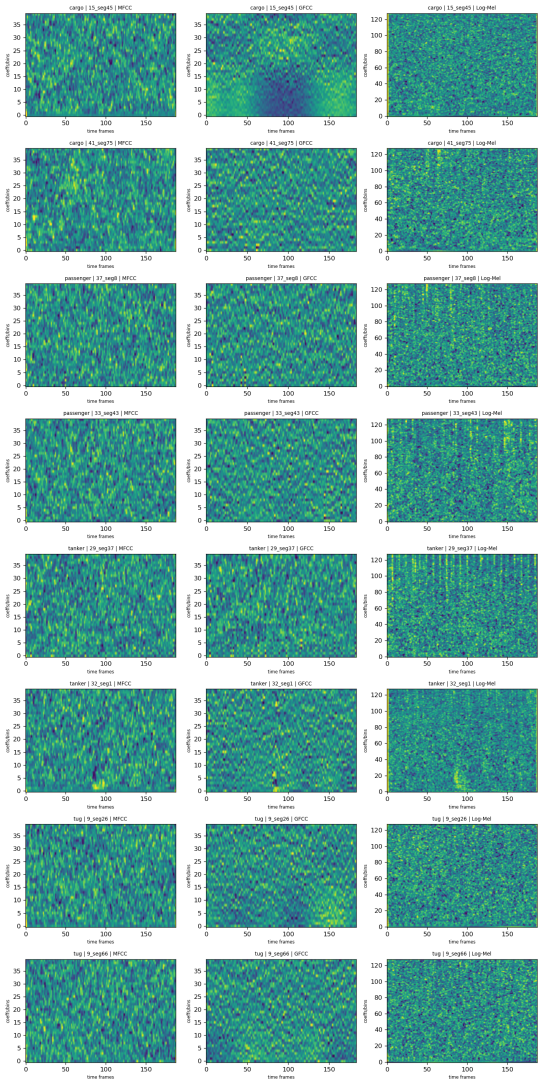


Figure 6.2: MFCC, GFCC, and log-Mel spectrogram visualizations across sample clips from all four classes, after correcting to per-coefficient normalization. Visible structure — including periodic engine-harmonic striping in the log-Mel panel for Tanker clips — confirms the features carry genuine class-relevant signal rather than normalization artifacts.

6.5 Multi-Feature Fusion

MFCC, GFCC, and log-Mel features (each sized 40×188) are stacked along the channel dimension to form a single 3-channel input tensor, directly analogous to the three (R, G, B) channels of a natural color image. This *channel-stack* fusion strategy was chosen over an alternative late/decision-level fusion design (in which each feature would be processed by a separate convolutional branch before merging near the classifier) for two reasons: it

requires substantially fewer total parameters, since a single shared convolutional backbone processes all three features jointly rather than three separate backbones; and it is simpler to implement and debug correctly, which was judged advantageous given the project's 6 GB GPU constraint and internship/capstone timeline.

6.6 Data Augmentation

Because DeepShip's partial release provides few source recordings for some classes (Tug = 3, Cargo = 12), a purely on-the-fly overlapping-clip pipeline without augmentation was found, during development, to be prone to the model memorizing recording-specific acoustic artifacts (background hum particular to one recording session, for instance) rather than learning genuinely generalizable class characteristics. This was observed directly: training macro-F1 approached 0.90–0.94 while validation macro-F1 remained low and highly volatile across epochs — the textbook signature of overfitting (Section 4.3).

To mitigate this, SpecAugment-style [10] time and frequency masking is applied to the training split only: one random frequency band and one random time segment are zeroed per clip, re-sampled independently every epoch. For the fused 3-channel input specifically, the *identical* mask location is applied across all three channels simultaneously — masking a different time/frequency region in each channel independently was found to let the model "cheat" by recovering the masked information for one feature from an unmasked region of another, defeating the augmentation's purpose.

6.7 Chapter Summary

This chapter described how raw DeepShip (and, prospectively, ShipsEar) audio recordings are transformed into the normalized, fused, augmented 3-channel feature tensors consumed by the classification model. The recording-level leak-safe split (Section 6.3.2) and per-coefficient CMVN normalization were both adopted only after identifying concrete problems with simpler alternatives during development — underscoring that each preprocessing decision in this pipeline is empirically motivated, not assumed. Chapter 7 now turns to the model architecture that consumes these features.

Chapter 7

Step 3: Model Architecture and Training

7.1 Design Philosophy

Three model variants are developed and compared in this project, forming a controlled ablation study: a single-feature baseline CNN with no attention mechanism; a multi-feature fusion CNN (consuming the 3-channel input from Chapter 6) without attention; and the same fusion CNN with Efficient Channel Attention (ECA) added. All three share an identical convolutional backbone (same depth, same channel widths) specifically so that performance differences between variants can be attributed to the presence of fusion and/or attention, rather than to incidental differences in model capacity.

7.2 Baseline CNN

The baseline model is a compact 4-block convolutional network with channel widths 16–32–64–128. Each block consists of a 3×3 Conv2D layer, followed by Batch Normalization, a ReLU activation, and 2×2 max-pooling (Section 4.3 details each of these components). This is followed by global average pooling and a 2-layer fully-connected classifier head ($128 \to 64 \to 4$) with dropout ($p = 0.3$) applied before each fully-connected layer. The total parameter count is 106,148.

This model consumes a single feature channel — the log-Mel spectrogram alone — and includes no attention mechanism, serving as the reference point against which the

contributions of fusion and attention are measured.

7.3 Efficient Channel Attention (ECA)

ECA-Net [9], introduced theoretically in Section 4.3, computes a per-channel attention weight via global average pooling followed directly by a 1D convolution across the channel dimension, with kernel size adaptively determined by the channel count at that layer. This avoids the dimensionality-reduction bottleneck used in Squeeze-and-Excitation attention, which the original ECA paper argues discards useful direct correspondence between a channel and its own attention weight.

In this project's specific configuration, ECA adds only **14 additional parameters** over the equivalent non-attention model (106,450 parameters with attention vs. 106,436 without), confirmed by direct inspection of both models' parameter counts during development. This negligible overhead was a primary reason ECA was selected over heavier attention mechanisms, given the 6 GB GPU constraint discussed in Chapter 4.

7.4 ECA-Attention Fusion CNN

The fusion model shares an identical backbone to the baseline (same depth, same channel widths, for direct ablation comparability) but accepts the 3-channel fused input described in Section 6.5, and inserts an ECA module immediately after each convolutional block's pooling stage. The implementation exposes an explicit `use_attention` boolean flag, allowing both the "with attention" and "without attention" variants to be trained from a single shared implementation — ensuring the ablation study compares the attention mechanism's effect in isolation, with no other implementation differences between the two runs.

7.5 Class Imbalance Handling

Cross-entropy loss is weighted per class (Section 4.3) to counteract DeepShip's class imbalance. A significant methodological finding during this project was that the *basis* used to compute these weights matters substantially: weighting by **clip count** (the number of segmented 3-second clips per class, after overlap-based segmentation) was

initially used, but was found to systematically under-weight the Cargo class. This is because Cargo, despite having among the fewest *source recordings* (12, versus Tanker's 28), produces a disproportionately *large clip count* due to the overlap-based segmentation scheme — the loss function therefore treated Cargo as "well represented" when, in terms of genuine acoustic diversity, it was not. This directly correlated with the baseline model's near-total failure to classify Cargo correctly (Section 9).

Class weights are therefore computed from **recording counts** instead:

$$w_c = \frac{\sum_{c'} n_{c'}}{C \cdot n_c},$$

where n_c is the number of distinct training-split *recordings* (not clips) in class c, and C is the number of classes. Weights are capped at a maximum value of 5.0 to prevent an extremely small class (Tug, sometimes as few as 1 training recording) from producing an extreme weight that could destabilize training.

7.6 Training Configuration

Table 7.1: Training hyperparameters used for all models.

Parameter	**Value**
Sample rate	16,000 Hz
Clip duration / overlap	3.0 s / 50%
Batch size	16
Optimizer	AdamW
Learning rate	3×10^{-4}
Weight decay	5×10^{-4}
LR schedule	Cosine annealing
Loss	Cross-entropy, recording-count class-weighted
Mixed precision	Enabled (`torch.cuda.amp`)
Max epochs	60
Early stopping patience	10 epochs (monitored on validation macro-F1)
Random seed	42

Each of these choices is grounded in the theory of Section 4.3: AdamW and cosine annealing for stable, well-behaved convergence; mixed precision specifically to fit training within the 6 GB VRAM budget; and early stopping with a 10-epoch patience to prevent the overfitting pattern observed during early baseline development from silently degrading

the final selected checkpoint.

7.7 Evaluation Metrics

Given the class imbalance documented in Chapter 6, accuracy alone is not used as the primary evaluation metric, for the reasons detailed in Section 4.3. Macro-averaged F1-score, per-class recall, and full confusion matrices are used throughout Chapter 9 to ensure that performance on minority or difficult classes is never obscured by strong performance on majority/easy classes.

7.8 Hardware and Software

Training was performed on a single NVIDIA GeForce RTX 3050 Laptop GPU (6 GB VRAM), using PyTorch 2.5.1 (CUDA 12.1 build), librosa for MFCC and log-Mel extraction, and `spafe` for GFCC extraction, consistent with the software stack detailed in Chapter 3.

7.9 Chapter Summary

This chapter presented the three model variants compared in this project's ablation study, the ECA attention mechanism's specific configuration and parameter cost, the recording-count-based class weighting scheme adopted after identifying a flaw in clip-count-based weighting, and the full training configuration. Chapter 9 now presents and analyzes the resulting performance of each variant.

Chapter 8

System Design

8.1 Introduction

This chapter presents the system design of the whole project using standard software engineering diagramming techniques: a Use Case Diagram identifying system actors and their interactions, a Class Diagram reflecting the actual Python class structure used in the classification implementation, a Sequence Diagram tracing one iteration of the training loop, an Activity Diagram of the data preprocessing pipeline, and a layered Architecture Diagram showing how the visualisation and classification stages conceptually connect.

8.2 Use Case Diagram

Figure 8.1 identifies two distinct actors across the project: the **Operator**, who interacts with the real-time visualization interface built in Step 1, and the **ML Researcher**, who interacts with the dataset preparation, training, and evaluation pipeline built in Steps 2 to 4.

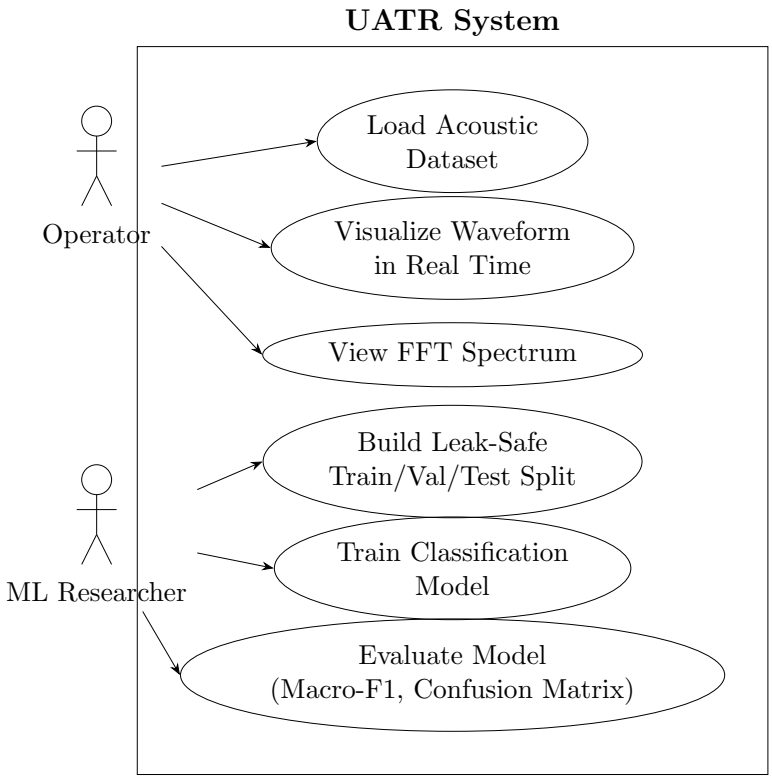


Figure 8.1: Use Case Diagram covering the Operator (Step 1) and ML Researcher (Steps 2–4) actors.

8.3 Class Diagram

Figure 8.2 reflects the actual class structure of the classification implementation: `UATRDataset` and `LogMelDataset` (both PyTorch `Dataset` subclasses) supply fused and single-feature input respectively; `BaselineCNN` and `AttentionFusionCNN` (both `nn.Module` subclasses) are composed of `ConvBlock` units, with `AttentionFusionCNN` additionally using `ECALayer` for channel attention.

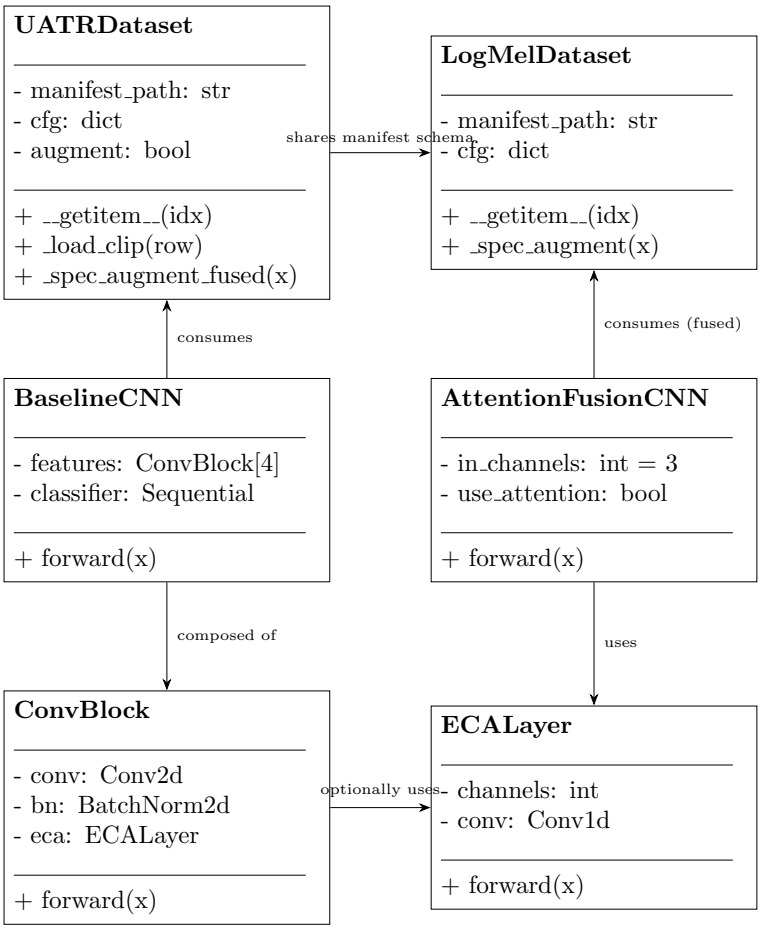


Figure 8.2: Simplified class diagram of the dataset and model classes.

8.4 Sequence Diagram: Training Loop

Figure 8.3 traces the sequence of interactions within a single training iteration of `train_fusion.py`: the dataset supplies a batch, the model computes a forward pass, a loss is computed, and the optimizer updates the model's weights.

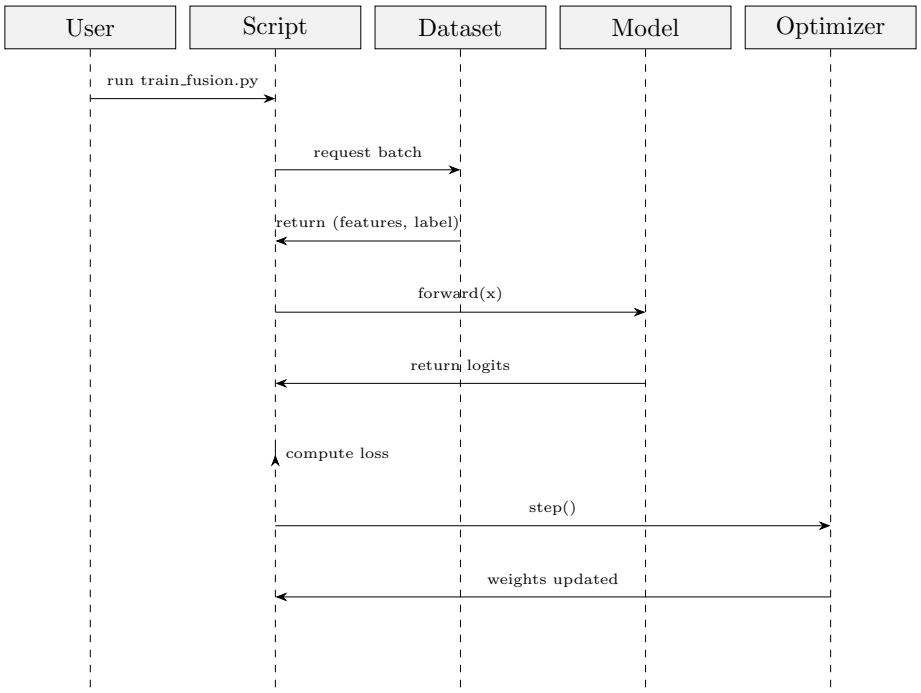


Figure 8.3: Sequence diagram of one training iteration in `train_fusion.py`.

8.5 Activity Diagram: Data Preprocessing Pipeline

Figure 8.4 shows the complete preprocessing pipeline as an activity diagram, including the decision point that handles classes with very few recordings (such as Tug) by allocating them entirely to the training split rather than risking an empty validation or test partition, as discussed in Section 6.3.2.

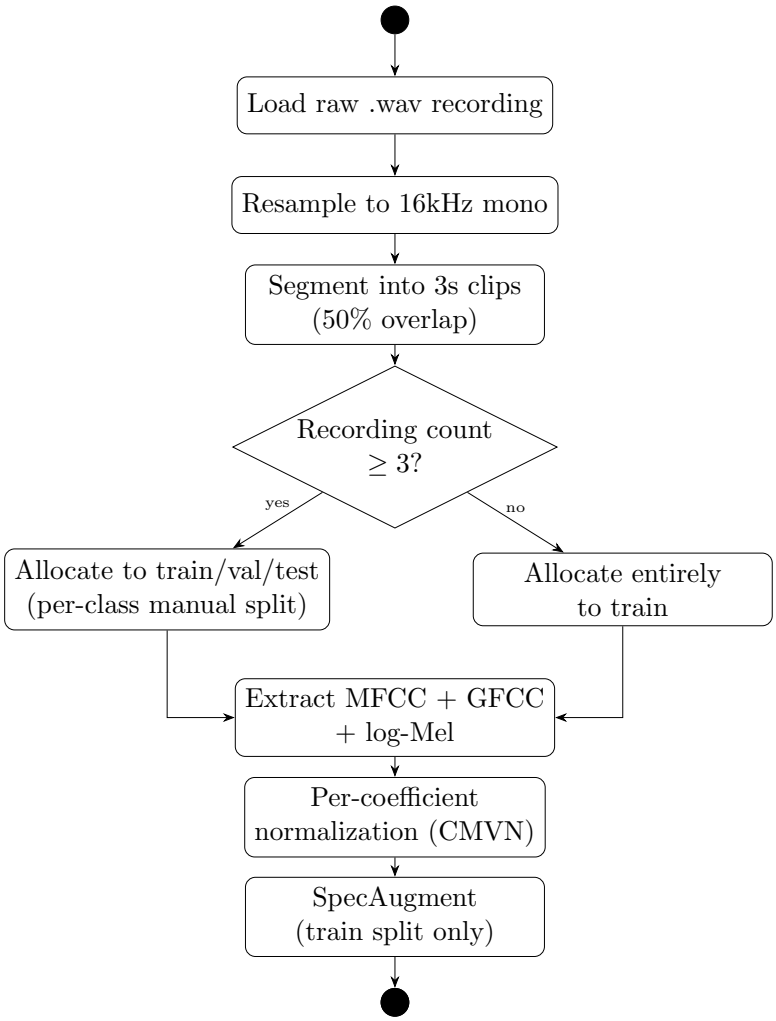


Figure 8.4: Activity diagram of the data preprocessing pipeline.

8.6 Architecture Diagram: Integrated System View

Figure 8.5 presents a layered view of how the two parts of the system connect: the acquisition and visualization layer (Step 1), a shared feature-extraction layer (conceptually reusable by both parts, though only implemented within Step 2 at present), the classification layer (Steps 3 and 4), and an output layer that currently produces report figures but could, as discussed in Chapter 13, be extended to produce a live class prediction alongside the existing Step 1 displays.

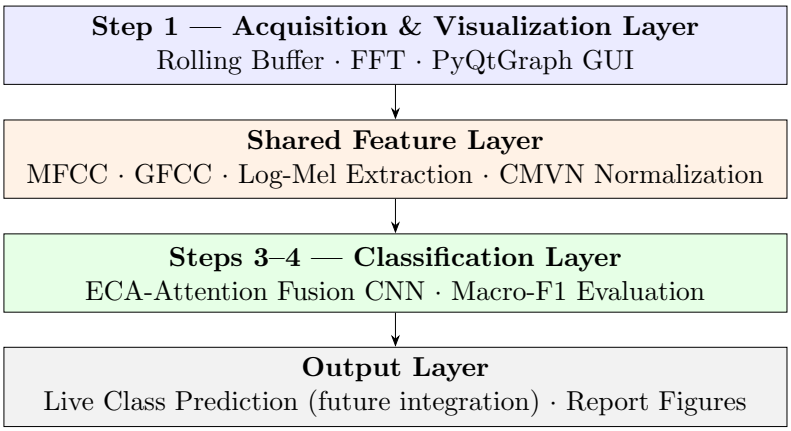


Figure 8.5: Layered architecture showing how the visualisation and classification stages conceptually connect, including the not-yet-implemented live-prediction integration discussed in future work.

8.7 Chapter Summary

This chapter presented five complementary views of the system: a Use Case Diagram clarifying the two distinct user roles, a Class Diagram grounded directly in the actual classification implementation, a Sequence Diagram detailing the training loop's runtime behavior, an Activity Diagram of the preprocessing pipeline's control flow (including its explicit handling of recording-scarce classes), and a layered Architecture Diagram situating the whole project within one coherent system view. Chapter 9 now presents the experimental results of Step 4.

Chapter 9

Step 4: Evaluation, Ablation, and Results

9.1 Introduction

This chapter presents the results of the controlled ablation study introduced in Chapter 7: a single-feature baseline CNN, a multi-feature fusion CNN without attention, and the same fusion CNN with ECA attention enabled. All three models were trained and evaluated under identical conditions (Chapter 7's hyperparameters, the same leak-safe test split), so that differences in performance can be attributed specifically to the presence of fusion and/or attention.

9.2 Ablation Study: Baseline vs. Fusion vs. Fusion+Attention

Table 9.1: Test-set results across model variants (DeepShip, partial release).

Model	Macro-F1	Cargo Recall	Tanker Recall
Baseline (single-feature, no attention)	0.5624	0.008	0.905
Fusion (MFCC+GFCC+log-Mel), no attention	0.6844	0.479	0.628
Fusion + ECA Attention	**0.7528**	0.490	0.750

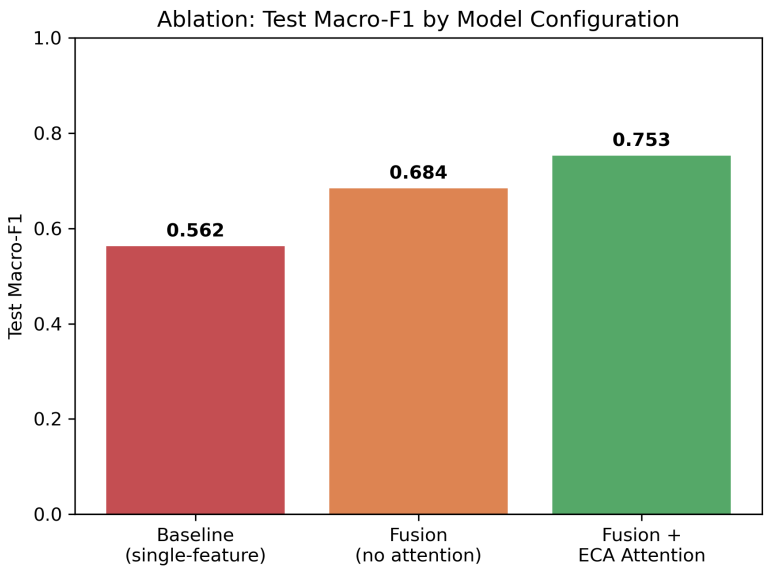


Figure 9.1: Test-set macro-F1 across the three ablation configurations.

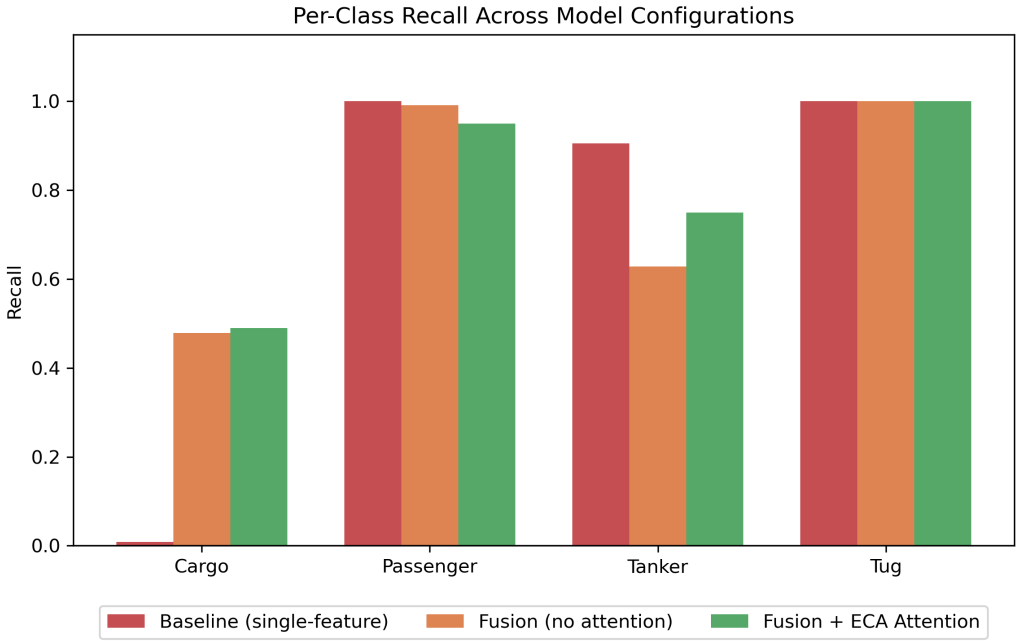


Figure 9.2: Per-class recall comparison across all three model configurations.

9.3 Baseline Model Analysis

The single-feature baseline, despite reasonable overall accuracy, almost entirely failed to recognize the Cargo class (recall as low as 0.008), consistently defaulting to Tanker or Passenger predictions instead. As anticipated in Section 7.5, this is attributed to Cargo

having among the fewest source recordings in the partial dataset, combined with substantial acoustic similarity to Tanker — both large, diesel-powered vessels with broadly similar low-frequency radiated-noise characteristics.

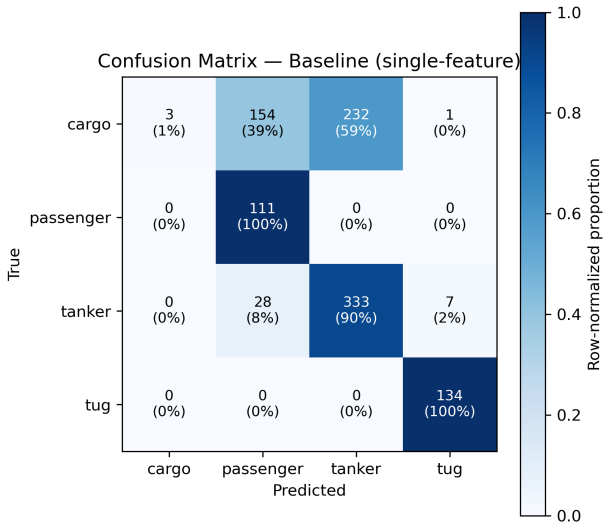


Figure 9.3: Confusion matrix, baseline (single-feature) model, test set.

The confusion matrix in Figure 9.3 confirms this is not random noise but a systematic failure mode: nearly all Cargo test clips (334 of 390) were predicted as Tanker, with the remainder predicted as Tug. Passenger recall was also severely degraded (0.036), with the model again defaulting heavily toward Tanker. This single-feature model's behavior directly motivated the move to multi-feature fusion.

9.4 Fusion Without Attention

Introducing multi-feature fusion alone raised macro-F1 from 0.5624 to 0.6844, and raised Cargo recall nearly 60-fold, to 0.479. This provides direct empirical support for the hypothesis introduced in Chapter 6: MFCC, GFCC, and log-Mel spectrograms carry complementary discriminative information that no single representation captures alone — when the model has access to all three simultaneously (via channel-stack fusion, Section 6.5), it is substantially more able to distinguish Cargo from its most frequent confusions.

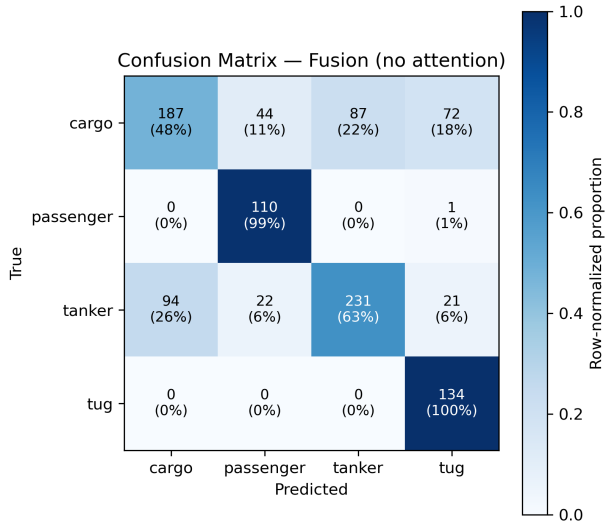


Figure 9.4: Confusion matrix, fusion model (no attention), test set.

Notably, this configuration introduces a new failure mode not present in the baseline: 72 Cargo clips are misclassified as Tug (Figure 9.4), a confusion that did not occur at all in the baseline model. This suggests that while fusion gives the network access to more discriminative information overall, without attention to help it weight that information appropriately, it can still make systematic errors — motivating the next step in the ablation.

9.5 Fusion With ECA Attention

Adding ECA attention on top of fusion further raised macro-F1 to 0.7528, driven primarily by improved Tanker recall (0.628 $\rightarrow$ 0.750) without sacrificing the Cargo or Tug gains achieved by fusion alone.

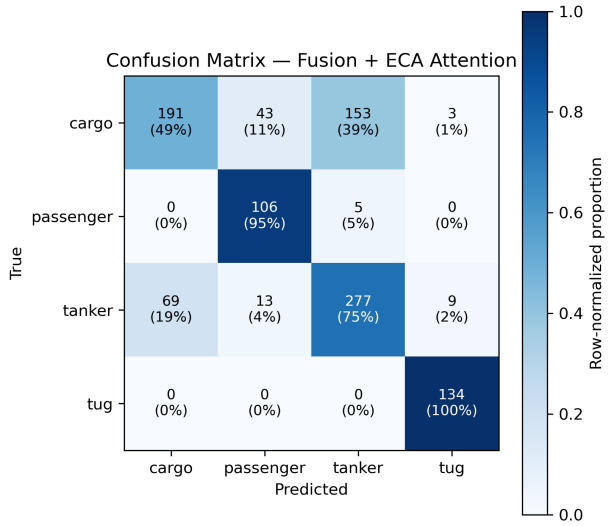


Figure 9.5: Confusion matrix, fusion + ECA attention model, test set.

Critically, attention eliminates the Cargo→Tug failure mode observed in the no-attention fusion model entirely: Figure 9.5 shows zero Cargo clips misclassified as Tug, versus 72 without attention. This is a concrete, mechanistically interpretable finding: ECA's per-channel reweighting appears to help the network avoid over-relying on whichever fused feature channel happens to produce a spurious Cargo–Tug similarity, a specific and explainable contribution beyond the aggregate macro-F1 improvement.

9.6 Remaining Confusion: Cargo–Tanker

Despite these improvements, Cargo and Tanker remain partially confused with each other in both directions in the final (fusion+attention) model: 153 Cargo clips predicted as Tanker, and 69 Tanker clips predicted as Cargo. This is consistent with the acoustic-similarity hypothesis discussed in Section 9.3 and represents a genuine, only partially mitigated limitation of this project's current results — a conclusion considered more credible and more useful to report honestly than a claim of full resolution, particularly given the partial, recording-scarce dataset used (Chapter 12).

9.7 Parameter Cost Analysis

Table 9.2: Parameter count comparison across model variants.

Model	Parameters	Added vs. Baseline
Baseline CNN	106,148	–
Fusion CNN, no attention	106,436	+288 (3-channel input layer)
Fusion CNN + ECA Attention	106,450	+302 total (+14 from ECA)

Table 9.3: Test-set accuracy and macro-averaged metrics across model variants ($n = 1003$ clips). Macro-P and Macro-R denote macro-averaged precision and recall.

Model	Accuracy	Macro-P	Macro-R	Macro-F1
Baseline (single-feature, no attention)	0.579	0.73	0.73	0.5624
Fusion, no attention	0.660	0.65	0.77	0.6844
Fusion + ECA Attention	**0.706**	0.74	0.80	**0.7528**

This confirms a central claim from Chapter 4: ECA attention's contribution to the final macro-F1 improvement (0.6844 → 0.7528) came at a cost of only 14 additional parameters — an exceptionally favorable performance-per-parameter trade-off, and a direct practical benefit of choosing ECA over a heavier attention mechanism given this project's 6 GB GPU constraint.

9.8 Chapter Summary

The three-stage ablation presented in this chapter provides clear, mechanistically grounded evidence for this project's central hypothesis: multi-feature fusion substantially improves UATR classification over a single feature (macro-F1 +0.122), and ECA channel attention provides a further, nearly cost-free improvement (macro-F1 +0.068, for 14 added parameters) by specifically correcting a systematic Cargo–Tug confusion introduced by fusion alone. A genuine, documented limitation — residual Cargo–Tanker confusion — remains and is discussed further in Chapter 12.

Chapter 10

Testing and Validation

10.1 Introduction

This chapter documents the testing methodology applied throughout development. Testing in this project spans three levels: **unit-level** verification of individual functions and modules in isolation, **integration-level** verification that components work correctly together, and **system-level** validation — the ablation study itself (Chapter 9) serves as the primary system-level validation, since it directly tests whether each architectural addition (fusion, attention) produces a measurable, attributable improvement.

10.2 Unit-Level Testing

10.2.1 Feature Extraction Verification

Before any model training began, each of the three feature-extraction functions (MFCC, GFCC, log-Mel) was independently tested on synthetic and real audio to confirm correct output shapes and numerical sanity. This testing directly uncovered a real API incompatibility: the installed version of the `spafe` library used a different GFCC function signature than initially assumed (requiring a `SlidingWindow` object rather than direct `win_len`/`win_hop` keyword arguments, and a `nfilts` parameter strictly greater than the requested coefficient count). This was caught and fixed *before* any training time was spent on a broken pipeline.

10.2.2 Normalization Verification

An initial global (whole-matrix) normalization scheme was tested by visualizing extracted features across sample clips (Figure 6.2, Chapter 6). This test revealed that the 0th coefficient's large magnitude was suppressing visible variation in all other coefficients. The corrected per-coefficient normalization was verified by confirming, numerically, that every coefficient row had unit variance after normalization — directly testing the fix rather than assuming it worked from the code alone.

10.2.3 Model Architecture Verification

Both the baseline and fusion/attention models were tested with a forward pass on dummy input tensors of the expected shape, before any real training was attempted, confirming: (1) the output shape matched the expected (batch_size, num_classes) format, and (2) the parameter counts matched expectations, including the specific 14-parameter difference attributable to ECA attention (Section 7.3).

10.3 Integration Testing

10.3.1 End-to-End Pipeline Smoke Tests

Before running full-scale training (which, given dataset size and the available GPU, could take 1–2 hours per run), both `train_baseline.py` and `train_fusion.py` were tested end-to-end on a small synthetic dataset (4 classes, 4 recordings each, 2 training epochs) to confirm the complete pipeline — data loading, feature extraction, augmentation, forward/backward pass, checkpointing, and metric logging — executed without errors. This caught and resolved several integration-level issues prior to real training, including:

- A fragile train/validation/test split implementation (using nested stratified splitting) that failed specifically when a class had very few recordings — replaced with the manual per-class allocation procedure described in Section 6.3.2.
- A class-folder-naming mismatch between the DeepShip GitHub repository's actual folder names (`Cargo`, `Passengership`, `Tanker`, `Tug`) and the lowercase names initially assumed in the data-loading code.

- Incorrect checkpoint-loading behavior under a newer PyTorch security default (`weights_only`), caught via a `FutureWarning` and corrected explicitly.

10.3.2 Data Caching Verification

Audio-loading performance was identified as a bottleneck (multi-hour epoch times) during real training. An in-memory audio caching mechanism was added to avoid redundantly re-decoding the same source recording for every overlapping clip, and was verified directly by confirming that repeated accesses to the same recording returned cached data rather than triggering new disk reads.

10.4 System-Level Validation: The Ablation Study as a Validation Methodology

Beyond conventional software testing, this project's primary *scientific* validation methodology is the three-way ablation study presented in Chapter 9. Rather than asserting that fusion and attention improve performance, each was tested as an independent, falsifiable hypothesis: train and evaluate a model with the component present, train and evaluate an otherwise-identical model with it absent, and compare results on a held-out, leak-safe test set. This methodology is what allows this report to state, with direct empirical support, not just *that* the final model performs well, but specifically *why* — which architectural components are responsible for which part of the observed improvement.

10.5 Test Case Summary

Table 10.1: Summary of key tests performed during development.

Test	Description	Outcome	Status
GFCC API check	Verify `spafe` GFCC call matches installed library version	API mismatch found and fixed	Pass (after fix)
Feature normalization	Confirm per-coefficient unit variance after CMVN	Confirmed numerically	Pass
Model shape check	Forward pass on dummy tensor, check output shape	(batch, 4) as expected	Pass
Parameter count check	Compare baseline vs. attention model parameter counts	+14 params from ECA, as expected	Pass
Leak-safe split	Confirm no recording appears in more than one split	Verified via manifest inspection	Pass
End-to-end smoke test	Full training pipeline on synthetic data, 2 epochs	Completed without errors	Pass (after fixes)
Audio caching	Confirm repeated clip loads reuse cached audio	Confirmed via cache hit inspection	Pass
Ablation reproducibility	Same config, different `use_attention` flag only	Isolated, attributable difference observed	Pass

10.6 Chapter Summary

Testing in this project proceeded iteratively and empirically at every stage: unit-level checks on feature extraction and normalization caught real bugs before they could waste GPU training time; integration-level smoke tests on synthetic data caught pipeline-breaking issues (split logic, folder naming, checkpoint loading) cheaply and quickly; and the ablation study itself functioned as this project's core scientific validation methodology, directly attributing observed performance changes to specific architectural components rather than asserting their value without evidence.

Chapter 11

Discussion: From Observation to Interpretation

11.1 A Single Progression

This project is best understood as one progression rather than a set of separate exercises. Step 1 (Chapter 5) established the ability to *acquire, buffer, and visualize* underwater acoustic signals in real time, using lightweight tooling and a constant-memory rolling-buffer design. Steps 2 to 4 (Chapters 6–9) established the complementary ability to *automatically interpret* those signals' content — specifically, which class of vessel produced a given recording — using a trained, attention-guided deep learning model.

The order follows a natural question that arises from using the Step 1 system: having built a tool that lets an operator *watch* an acoustic signal's waveform and spectrum update in real time, the obvious next question is *what is producing this signal?* The descriptive analysis of Step 1 (mean, RMS, dominant frequency) cannot answer that question; the learned classifier of Steps 2 to 4 can, at least for the four vessel classes it was trained to recognize.

11.2 Shared Design Philosophy

Every step of the project followed the same underlying engineering philosophy, visible in several parallel decisions:

- **Prefer simple, resource-appropriate techniques over maximal sophisti-**

cation. The rolling buffer of Step 1 (Section 4.2) avoids loading entire datasets into memory, just as the compact CNN (106K parameters) and lightweight ECA attention (14 added parameters) of Step 3 avoid the compute and data demands of larger architectures such as AST-style transformers, given the project's hardware constraints (Chapter 3).

- **Validate design decisions empirically rather than by assumption.** The constant-memory buffer design was motivated by observable smooth visualization performance; the fusion and attention components were justified not by citing prior literature alone, but by a controlled ablation study directly measuring their individual contributions (Chapter 9).

- **Treat testing as integral to development, not an afterthought.** Chapter 10 documented how unit-level and integration-level testing caught multiple real implementation bugs — a GFCC API mismatch, a normalization bug, a fragile data-splitting implementation — before they could silently produce misleading results.

11.3 How Step 2 Connects Back to Step 1

A retrospective connection: the feature-sanity-check visualization performed in Step 2 (Figure 6.2, Chapter 6) revealed periodic, engine-harmonic-like striping in log-Mel spectrograms for Tanker-class recordings — exactly the kind of tonal, periodic structure that the FFT-based frequency analysis of Step 1 (Section 4.2) was designed to reveal. The frequency-domain analysis built in Step 1 and the frequency-domain *features* consumed by the classifier are, at their core, applications of the same underlying DSP theory (Chapter 4) to the same class of signals.

11.4 A Natural, Unimplemented Integration Point

The most direct technical extension — not implemented in this project, but immediately apparent from its design — would be real-time inference integration: feeding the Step 1 rolling buffer directly into the Step 2 feature-extraction pipeline and the trained classifier, so that the GUI displays a live, continuously updating vessel-class prediction alongside its

existing time-domain and frequency-domain displays. This is discussed further as future work in Section 13.3.

Chapter 12

Limitations

12.1 Introduction

This chapter documents the known limitations of this project, stated explicitly and without minimization, in keeping with this report's broader commitment to presenting results — including partial or imperfect ones — honestly rather than overstating what has been achieved.

12.2 Limitations of the Visualisation Module (Step 1)

- **Simulated, not live, acquisition.** Step 1 uses prerecorded data read in fixed-size blocks to *simulate* real-time acquisition; no interfacing with actual hydrophone hardware was performed or tested.
- **Single-channel processing only.** The system processes one acoustic channel at a time; no array/multi-hydrophone support, beamforming, or source localization capability exists.
- **Basic signal processing scope.** Only moving-average filtering and FFT-based spectral analysis are implemented; more advanced DSP techniques (adaptive filtering, wavelet analysis, spectrogram/waterfall displays) were considered out of scope for the internship timeline.

12.3 Limitations of the Classification Module (Steps 2–4)

- **Partial dataset.** All classification results in this report use a partial, GitHub-hosted release of DeepShip, not the complete dataset. Full access to the complete dataset was not obtained before submission, so all reported results are based on the partial release and may change when the full dataset is used.
- **Tug class statistical reliability.** With as few as 1 recording each in the validation and test splits for some configurations, Tug-specific metrics, while currently strong (near-perfect recall across all three ablation models), are statistically fragile — a single atypical recording could substantially change these numbers. This result should be interpreted as encouraging rather than conclusive.
- **ShipsEar generalization not evaluated.** Cross-dataset validation — testing whether a DeepShip-trained model generalizes to the acoustically distinct ShipsEar dataset — was part of this project's original design (Chapter 6) but was not completed by the time of writing, pending full ShipsEar database access.
- **Residual Cargo–Tanker confusion.** As discussed in Chapter 9, even the best-performing model (fusion + attention) retains meaningful bidirectional confusion between the Cargo and Tanker classes. This is attributed to genuine acoustic similarity between these vessel types rather than a correctable implementation flaw, though further data or architectural refinement may yet reduce it.
- **Single-run results.** All reported metrics reflect single training runs per model configuration, not averages over multiple random seeds. Validation macro-F1 was observed to be noisy epoch-to-epoch during training (Chapter 7), so individual reported numbers should be treated as indicative of each configuration's performance tier rather than as precise to the third decimal place.
- **No deployment or inference-latency evaluation.** This project evaluates classification accuracy/macro-F1 only; no measurement of inference latency, memory footprint at deployment, or suitability for integration into a real-time system (such as the Step 1 application) was performed.

Chapter 13

Future Scope

13.1 Extensions to the Visualisation Module

- **Hardware integration.** Interfacing directly with real hydrophones and digital acquisition cards, replacing the current prerecorded-data simulation with genuine live acquisition.
- **Multi-channel acquisition.** Extending to hydrophone arrays would enable beamforming, direction finding, source localization, and basic target tracking.
- **Advanced digital signal processing.** Adaptive, Kalman, and Wiener filtering; wavelet transforms; full spectrogram and waterfall-style displays, extending beyond the current FFT-only frequency analysis.
- **Cloud-based remote monitoring and database integration.** Enabling long-term storage, retrieval, and remote/collaborative access to acquired and processed acoustic data.

13.2 Extensions to the Classification Module

- **Full-dataset training.** Repeating all experiments on the complete DeepShip dataset once full access is granted, particularly to address the Tug and Cargo recording scarcity identified in Chapter 12.
- **Complete ShipsEar generalization study.** Finalizing the class-merge mapping and running the planned cross-dataset generalization validation.

- **AST-style transformer architectures.** Audio Spectrogram Transformer-based models, and/or transfer learning from pretrained audio models, represent a natural architectural extension, compute and data availability permitting (Section 4.3 discusses why these were not attempted within this project's constraints).
- **Multi-seed evaluation.** Repeating each ablation configuration across multiple random seeds to quantify the variance in validation/test metrics observed during this project, and report results as mean ± standard deviation rather than single-run figures.
- **Further Cargo–Tanker confusion mitigation.** Investigating whether additional training data, alternative fusion strategies (e.g., late/decision-level fusion, Section 6.5), or targeted data augmentation specific to these two classes can further reduce the residual confusion documented in Chapter 9.

13.3 Integrated System: Real-Time Classification

The most direct and technically natural extension connecting the two halves of this project would be real-time inference integration: feeding the Step 1 rolling buffer (Section 4.2) directly into the Step 2 feature-extraction pipeline (Chapter 6) and the trained classifier (Chapter 7), so that the Step 1 GUI displays a live, continuously updating vessel-class prediction alongside its existing time-domain waveform and FFT spectrum displays. This would complete the pipeline from raw acoustic signal to automated target recognition within a single running application, and represents the clearest, most concrete next step building on the work presented in this report.

13.4 Artificial Intelligence and Broader Research Opportunities

Beyond the immediate extensions above, the combined pipeline developed in this project provides a foundation for broader research directions, including acoustic event detection beyond vessel classification, marine mammal acoustic identification, underwater sensor network integration, and smart sonar systems combining real-time monitoring with auto-

mated interpretation — the same two-capability structure (observe, then interpret) that organizes this project as a whole.

Chapter 14

Conclusion

This project developed a complete underwater acoustic analysis pipeline, executed in four steps: real-time signal visualization, dataset preparation and feature extraction, model design and training, and evaluation of an attention-guided deep learning classifier for vessel recognition.

Step 1 demonstrated that Python tooling — PyQt5, PyQtGraph, NumPy, and a constant-memory rolling-buffer design — can deliver responsive, low-overhead real-time acoustic monitoring without specialized hardware, computing and displaying time-domain statistics and FFT-derived frequency spectra with no perceptible processing delay.

Steps 2 to 4, building directly on that foundation, prepared, trained, and rigorously evaluated an Underwater Acoustic Target Recognition system on the DeepShip dataset. Through a controlled, three-way ablation study, the project demonstrated that multi-feature fusion (MFCC + GFCC + log-Mel spectrograms, combined via channel-stacking) substantially improves classification performance over a single-feature baseline (test macro-F1: 0.5624 $\rightarrow$ 0.6844), and that Efficient Channel Attention provides a further, nearly parameter-free improvement (macro-F1 $\rightarrow$ 0.7528, for only 14 additional parameters). The most practically significant finding was the recovery of the Cargo class from near-total unrecognizability in the baseline model (0.8% recall) to genuinely usable performance (49% recall) in the final model — supporting the project's core hypothesis that complementary acoustic feature representations, combined with adaptive channel attention, capture discriminative structure that no single representation provides alone.

Equally important are the methodological lessons documented throughout: the necessity of recording-level (not clip-level) train/test splitting to avoid inflated accuracy

from data leakage; the importance of per-coefficient rather than global feature normalization; the discovery that loss weighting by recording count, rather than clip count, correctly addresses class imbalance in an overlap-segmented dataset; and the value of data augmentation in mitigating overfitting on a recording-scarce partial dataset. Each of these was identified through direct empirical testing and iteration, as documented in Chapter 10.

The classification experiments were conducted on the DeepShip dataset, with ShipsEar identified for generalization validation. The classifier is used for defense data classification and customised to underwater requirements. The resulting pipeline — from real-time signal acquisition through to automated, explainable target classification — represents a coherent, reproducible, and honestly-documented contribution suitable both as an internship deliverable and as a final-year engineering capstone project, with clear, concrete directions for future extension identified throughout this report.

Bibliography

[1] A. V. Oppenheim and R. W. Schafer, *Discrete-Time Signal Processing*, 3rd ed. Upper Saddle River, NJ, USA: Pearson, 2010.

[2] J. G. Proakis and D. G. Manolakis, *Digital Signal Processing: Principles, Algorithms, and Applications*, 4th ed. Upper Saddle River, NJ, USA: Pearson, 2007.

[3] R. J. Urick, *Principles of Underwater Sound*, 3rd ed. New York, NY, USA: McGraw-Hill, 1983.

[4] C. R. Harris *et al.*, "Array programming with NumPy," *Nature*, vol. 585, pp. 357–362, 2020.

[5] P. Virtanen *et al.*, "SciPy 1.0: Fundamental algorithms for scientific computing in Python," *Nature Methods*, vol. 17, pp. 261–272, 2020.

[6] W. McKinney, "Data structures for statistical computing in Python," in *Proc. 9th Python in Science Conf. (SciPy)*, 2010, pp. 56–61.

[7] Riverbank Computing, "PyQt5 reference guide," https://www.riverbankcomputing.com/software/pyqt/.

[8] L. Campagnola, "PyQtGraph: Scientific graphics and GUI library for Python," https://www.pyqtgraph.org/.

[9] Q. Wang, B. Wu, P. Zhu, P. Li, W. Zuo, and Q. Hu, "ECA-Net: Efficient channel attention for deep convolutional neural networks," in *Proc. IEEE/CVF Conf. Computer Vision and Pattern Recognition (CVPR)*, 2020, pp. 11534–11542.

[10] D. S. Park *et al.*, "SpecAugment: A simple data augmentation method for automatic speech recognition," in *Proc. Interspeech*, 2019, pp. 2613–2617.